arXiv:2602.23471v1 [cs.IR] 26 Feb 2026

Cross-Representation Knowledge Transfer for Improved Sequential Recommendations

Artur Gimranov*
agimranov@hse.ru
HSE University
Moscow, Russian Federation

Viacheslav Yusupov*
vyusupov@hse.ru
HSE University
Moscow, Russian Federation

Elfat Sabitov*
a.sabitov@hse.ru
HSE University
Moscow, Russian Federation

Tatyana Matveeva*
tpmatveeva@hse.ru
HSE University
Moscow, Russian Federation

Anton Lysenko
av.lysenko@hse.ru
HSE University
Moscow, Russian Federation

Ruslan Israfilov
SB AI Lab
Moscow, Russian Federation

Evgeny Frolov
AXXX
HSE University
Moscow, Russian Federation

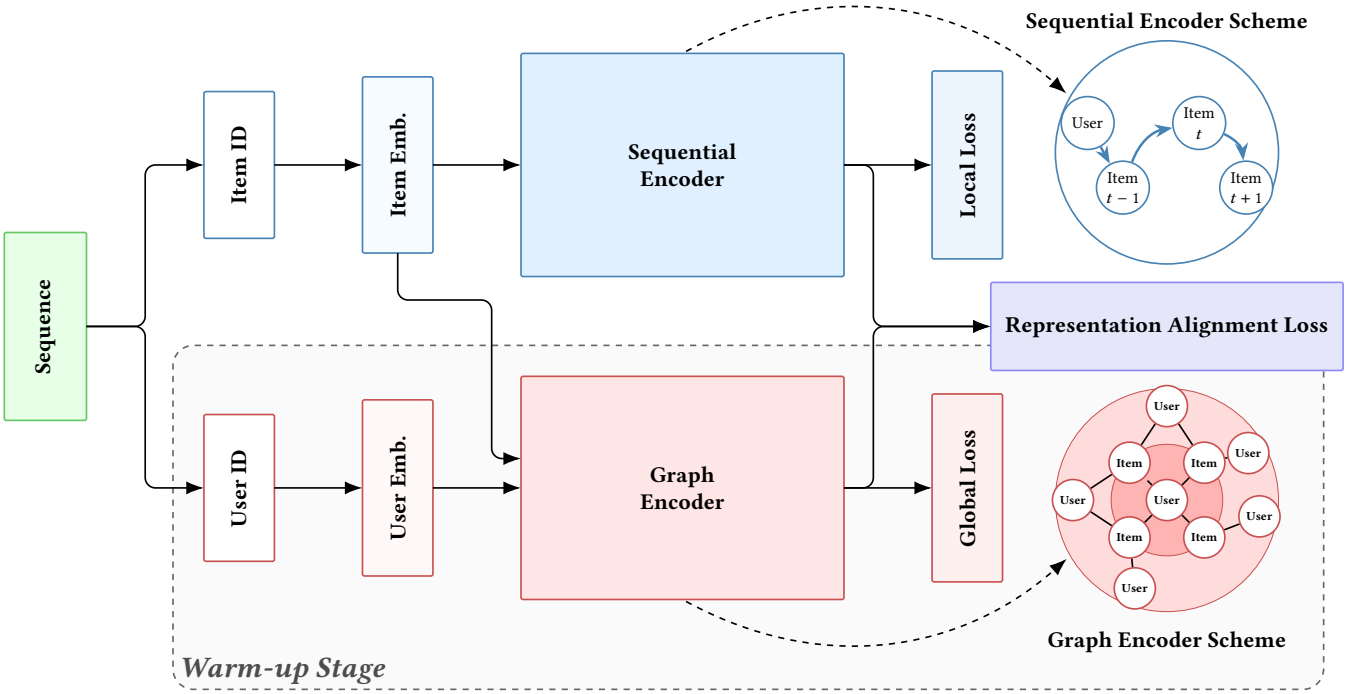

Figure 1: Overview of the CREATE framework. A sequential encoder (top) models users' local interaction dynamics, while a graph encoder (bottom) captures global item–item context. Their embeddings are aligned via a representation-alignment objective. Training proceeds in two phases: the graph encoder is first pre-trained during a warm-up stage, after which both encoders are jointly optimized end-to-end.

*Authors contributed equally to this research.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
KDD '26, Jeju, Korea
© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

Abstract

Transformer architectures, capable of capturing sequential dependencies in the history of user interactions, have become the dominant approach in sequential recommender systems. Despite their success, such models consider sequence elements in isolation, implicitly accounting for the complex relationships between them. Graph neural networks, in contrast, explicitly model these relationships through higher order interactions but are often unable to adequately capture their evolution over time, limiting their use for predicting the next interaction. To fill this gap, we present a new framework that combines transformers and graph neural networks and aligns different representations for solving next-item prediction task. Our solution simultaneously encodes structural dependencies in the interaction graph and tracks their dynamic change. Experimental results on a number of open datasets demonstrate that the proposed framework consistently outperforms both pure sequential and graph approaches in terms of recommendation quality, as well as recent methods that combine both types of signals.

CCS Concepts

• **Information systems → Recommender systems**.

Keywords

Recommender Systems, Sequential Learning, Graph Neural Networks, Representation Learning

ACM Reference Format:

Artur Gimranov, Viacheslav Yusupov, Elfat Sabitov, Tatyana Matveeva, Anton Lysenko, Ruslan Israfilov, and Evgeny Frolov. 2018. Cross-Representation Knowledge Transfer for Improved Sequential Recommendations. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (KDD '26).* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Recommender systems play a crucial role in modern digital platforms. They leverage available data on user–item interactions to rank items that a user is likely to prefer in the future. Recently, deep learning-based approaches have become increasingly popular [9, 47] dominating recommendation systems, particularly in the area of sequence-based recommendations [38] but not limited to them [6, 12, 45].

For recommendations on sequences where user interactions are modeled over time, state-of-the-art models, such as SASRec [15] and BERT4Rec [32], primarily utilize users' collaborative signal, considering each sequence of user items independently. This allows the model to capture the relationships between items based on their order, and build rich representations of both users and items. However, such models do not account for certain global relationships of items and users between each other.

However, depending on the domain, not all data is sequential [19], and such scenarios may require models that do not rely on the order of interactions. One such approach is the use of graph neural networks (GNNs), which model user-item interaction graph. Recent studies demonstrate the application of graph neural networks in recommendation tasks [1, 35], showing superiority over non-sequential models. However, GNNs do not account for the sequential nature of interactions, resulting in inferior recommendation quality compared to SOTA sequence-based models [24]. Thus, both sequence and graph approaches attempt to model user preferences by considering different data structures: local (a user's item sequence), and global context, expressed as a user subgraph, explicitly defining relationships between users and items.

Recent work has shown that using different interaction structures can improve the quality of recommendations in the next-item prediction task [2, 4, 21]. These studies combine two architectures: transformers and GNNs, where SASRec is used most often as the transformer model, and LightGCN [12] or other graph encoders are used as the graph model. Despite this, existing solutions have a number of significant drawbacks related to the way the resulting representations are fused, as well as the method for predicting new recommendations.

In our work, we present a Cross-REpresentation Aligned Transfer Encoders (CREATE) framework that combines transformer and graph architectures in a way that enriches the sequence-based model with global knowledge and produces representations aligned with each other (see Figure 1). It mitigates the problems of previous works: (i) recommendation updates are based on the sequential encoder's scheme, which avoids usage of user embeddings, making our approach asymmetric and independent from folding-in procedure [41], an approach for updating user embeddings in correspondence to new interactions; (ii) for representation alignment, Barlow Twins is used instead of contrastive loss, which shows better results and enables redundancy reduction. Thus, our contributions are:

- We propose a framework that combines transformer and graph architectures for the next item prediction task;
- We develop a new method of training a model under the framework that helps improve the quality of recommendations;
- We introduce a representation alignment method between encoders, which enables redundancy reduction and improves recommendation quality.

All the experiments are implemented in a global temporal split setting, which is a better approximation to the industrial setting than leave-last-out [11]. The rest of the paper is organized as follows: Sec. 2.1 contains detailed related work, Sec. 3.1 proposes our approach, Sec. 4.1 describes the experimental setup and results, and Sec. 5 presents the ablation study. Code implementation is publicly available at https://anonymous.4open.science/r/multirepr_recsys-761F.

2 Related work

2.1 Sequential recommendations

Transformer-based deep models for sequential recommendation [3] have become the dominant approach in the field due to their natural suitability for sequential data [19]. One of such model is SASRec [15], offers state-of-the-art performance. However, its original version uses binary cross-entropy as a loss function, which can lead to biased predictions and subsequent studies show that switching to full cross-entropy significantly improves results [10, 18, 27]. Because SASRec operates solely on the relative order of items

in a user's sequence, modifications have been proposed that enrich the model with additional information, such as time intervals between interactions [20], content [13, 22] and context [29, 40]. Another modern standard is a BERT4Rec [32], which in contrast to unidirectional attention, uses a bidirectional attention mechanism, masking out positive items from a sequence. Further developments to transformer-based architectures are aimed at applying self-supervised learning methods to the obtained representations: CL4SRec [36] uses contrastive learning for this purpose, while the Barlow Twins approach [30] achieves similar goals without the need for negative sampling.

2.2 Graph-based recommendations

Graph neural networks (GNNs) have become a powerful tool in this field because they directly exploit graph structure of user-item interactions [35]. This allows them to capture complex, high-level relationships and transitive dependencies that are difficult to detect when examining interactions sequentially or in isolation [1, 8, 39, 43]. Early studies applying GNNs to recommendation problems considered applying existing architectures to a bipartite graph of user-item interactions [35], including NGCF [33] and an attention version of graph neural network GAT [16]. Later work demonstrated that the general GCN [17] architecture itself can be excessive for recommendation systems, as reflected in LightGCN [12], which eschews fully connected neural networks for processing neighbor representations and instead trains only user and item embeddings, significantly improving recommendation quality. Later work UltraGCN [25] addresses the oversmoothing problem of a LightGCN by considering user and item representations in the limit, with an infinite number of hops on a graph. Some studies also consider item-item graphs [21, 25], the edges of which are constructed from data or learned during model training.

2.3 Multi-representational models

Multi-representational models [7, 46] are defined as models that use object representations from different sources (in our case, embeddings obtained using a sequential encoder and a graph encoder). Transformer architectures, as well as graph architectures, have fundamental limitations. Transformer architectures consider item sequences independently, explicitly failing to consider higher-order relationships. While addressing this shortcoming [37], GNNs fail to account for the sequential nature of the data, which clearly impacts the performance of next-item prediction tasks [21, 34]. Given this, combining these two types of information is a relevant approach that potentially improves recommendation quality, as confirmed by several studies described below.

Currently, there are few studies combining transformer and graph representations, as the field is new. Among the first studies, the MCLSR [21, 34] approach is noteworthy, implementing late fusion of representations obtained from sequential and graph parts, using a contrastive loss to align the representations. Another work that stands out among the late fusion approaches is MRGSRec [2], which, unlike the previous work, uses graph representations during prediction. In contrast, some works use an early fusion mechanism. For instance, GSAU [4] employs a shared embedding layer for both the transformer and graph components. It uses a contrastive loss function, designed to bring similar items closer and push dissimilar ones apart in the latent space [14], and the binary cross-entropy. However, later works demonstrates that replacing BCE loss with cross-entropy in transformer models improves performance [18, 27]. The most recent model of this type is LOOM [21], in which a sequential encoder serves as a teacher for a graph using item-item relationships, the weights of which are learned from the data. LOOM also utilizes late fusion of the embeddings obtained by the sequential encoder and the graph model. Here, information enrichment occurs unilaterally, and the user's graph representation is not updated when new data arrives, which may impact the quality of recommendations.

It is also worth noting that all the aforementioned works evaluated their quality using a next-item prediction approach using a leave-last-out partitioning scheme, which differs from the real-world scenario [11].

3 Proposed approach

This section describes the problem statement of a sequential recommendations and the proposed cross-representation knowledge transfer framework, which scheme is presented at Figure 1.

3.1 Problem statement

Let us first denote the set of users as $\mathcal{U}$, and the set of items as $\mathcal{I}$, where $|\mathcal{U}| = M$ and $|\mathcal{I}| = N$. The next-item prediction problem is formulated as predicting the next item $i^u_{|H_u|+1}$ for user $u \in \mathcal{U}$, given his history of past interactions $H_u = (i^u_1, i^u_2, \ldots, i^u_{|H_u|})$.

However, existing recommendation algorithms that use the graph structure of user interactions do not take into account the temporal order of items in the user's history, but rely on so-called global connections between users and items. By this, we can reformulate the problem for graph recommenders as: for a user $u \in \mathcal{U}$, predict the next item $i^u_{|H_u|+1}$ based on user context C^u, including the edges of a bipartite interaction graph $\mathcal{G}$ between users $\mathcal{U}$ and items $\mathcal{I}$.

Below are the components of our CREATE framework, which includes a shared embedding layer, sequential and graph encoders and the alignment component. The overall algorithm diagram is shown in Figure 1.

3.2 Embedding layer

We map user and item identifiers into a d-dimensional space using learnable lookup functions

$$U_e : \mathcal{U} \to \mathbb{R}^d, \qquad I_e : \mathcal{I} \to \mathbb{R}^d.$$

Thus, the representations of a user u and an item i are:

$$\mathbf{e}^u = U_e(u) \in \mathbb{R}^d, \qquad \mathbf{e}_i = I_e(i) \in \mathbb{R}^d,$$

the user's interaction history $H_u = (i^u_1, \ldots, i^u_c)$ can be mapped into latent space:

$$\mathbf{E}_u = (\mathbf{e}_{i^u_1}, \ldots, \mathbf{e}_{i^u_c}) \in \mathbb{R}^{c \times d}, \qquad \mathbf{e}_{i^u_k} = I_e(i^u_k),$$

Inheriting the logic of transformers, positional embeddings are used to encode ordinal information:

$$P_e : \{1, \ldots, c\} \to \mathbb{R}^d,$$

and form position-aware item representations by additive composition:

$$\hat{\mathbf{e}}_{i_k^u} = \mathbf{e}_{i_k^u} + P_e(k), \qquad \hat{\mathbf{E}}_u = (\hat{\mathbf{e}}_{i_1^u}, \ldots, \hat{\mathbf{e}}_{i_c^u}) \in \mathbb{R}^{c\times d}.$$

This choice is standard in sequential recommenders with learnable positional encodings [15, 32].

3.3 Sequential encoder

As the sequential encoder in CREATE, we can plug in a broad class of sequence models, including RNN/CNN-based architectures and, more commonly, Transformer-based recommenders. In this work, we utilized a widely used SASRec encoder with cross-entropy loss [18, 27], which shows the state-of-the-art performance on a broad range of sequential datasets [19]. In addition, we also conducted experiments with BERT4Rec [32] as a sequential backbone. Where the full CE is not applicable due to the large catalog size, we utilize scalable cross-entropy loss [27] to reduce the computational cost of the model.

SASRec. For each user u, we take the learnable item embeddings from the shared embedding layer (augmented with positional information) and feed them into a causal Transformer encoder, in our case this is SASRec. The uni-directional self-attention ensures that the representation at position t only attends to items $1, \ldots, t$, matching the next-item prediction setup. Denote the encoder output by

$$\mathbf{H}_u^{\text{SASREC}} = \text{TransformerEncoder}\left(\hat{\mathbf{E}}_u\right) \in \mathbb{R}^{c\times d},$$

and let $\mathbf{h}_u \in \mathbb{R}^d$ be the user-state vector used for next-item prediction (e.g., the output at the last position):

$$\mathbf{h}_u = \mathbf{H}_u^{\text{SASREC}}[c].$$

BERT4Rec. Alternatively, BERT4Rec [32] uses a bidirectional Transformer encoder trained via masked item prediction. Unlike causal masking, the self-attention mechanism has access to all positions in the sequence, using both past and future interactions. During training, we randomly select a subset of positions $\mathcal{M} \subset \{1, \ldots, c\}$ and replace their corresponding item embeddings with a learned `[MASK]` embedding $\mathbf{e}_{\text{MASK}} \in \mathbb{R}^d$. The bidirectional encoder processes the entire modified sequence:

$$\mathbf{H}_u^{\text{BERT}} = \text{BidirectionalTransformer}\left(\tilde{\mathbf{E}}_u\right) \in \mathbb{R}^{c\times d},$$

At inference time, we append an extra `[MASK]` token at last position and take its output representation as the user-state vector:

$$\mathbf{h}_u = \mathbf{H}_u^{\text{BERT}}[c].$$

Prediction head and training objective. Given a predicted user-state vector $\mathbf{h}_u$ from a transformer, we compute scores for all items via a dot product with the item embedding matrix $\mathbf{W}_I \in \mathbb{R}^{N\times d}$ (typically the same learnable embedding table as I_e):

$$\mathbf{s}_u = \mathbf{W}_I \mathbf{h}_u \in \mathbb{R}^N, \qquad s_{u,i} = \langle \mathbf{h}_u, \mathbf{e}_i \rangle,$$

where $\mathbf{e}_i$ is the embedding of item i.

For training, we use the full softmax cross-entropy objective over the entire item set:

$$\mathcal{L}_{\text{full}} = -\sum_{(u,t)} \log \frac{\exp\left(s_{u,i_{t+1}^u}\right)}{\sum_{j\in I} \exp\left(s_{u,j}\right)},$$

where (u, t) ranges over training positions, and i_{t+1}^u denotes the ground-truth next item.

3.4 Graph encoder

The graph backbone in CREATE operates on the user-item graph of interactions, sharing item embeddings with the sequential backbone, capturing global interaction patterns. In particular, user embeddings are utilized only during training; inference relies solely on the sequential backbone with enriched item embeddings. This design eliminates the need for a "folding-in" procedure [41] to update user representations for new interactions. Consequently, CREATE remains robust to out-of-training user activity and supports asymmetric graph structures [5] that prioritize item-item relations over persistent user states.

As graph backbone we used LightGCN [12] and UltraGCN [25]. LightGCN is a simplified Graph Convolution model, which omits operations such as self-connection, feature transformation, and nonlinear activations. UltraGCN further simplifies Graph Convolution and considers the limit case of message passing for an infinite amount of layers.

Regarding the training process, LightGCN optimizes the Bayesian Personalized Ranking (BPR) loss [31]: the difference between positive $s^u_{i\in H_u}$ and negative scores $s^u_{j\notin H_u}$ of the graph model

$$L_{BPR} = -\sum_{u\in\mathcal{U}} \sum_{i\in H_u} \sum_{j\notin H_u} \ln \sigma(s_i^u - s_j^u).$$

Optionally, the regularization term can be added $\lambda_{reg}(\sum_{u\in\mathcal{U}} \|\mathbf{e}^u\| + \sum_{u\in\mathcal{U}, i\in H_u} \|\mathbf{e}_i\|)$, which regulates the $L2$ norms of the user and item embeddings, respectively.

The UltraGCN model [25] optimizes the loss function consisting of three parts:

$$L = L_O + \lambda L_C + \gamma L_I, \tag{1}$$

where L_O stands for the general objective, which is Binary Cross-Entropy (BCE) for link prediction with uniform weighting and hard negatives; L_C is a constraint component, which helps UltraGCN approximate infinite-layer propagation and L_I stands for item-item constraint.

3.5 Representation alignment

Our model forms two views of each user: a sequential (*local*) representation capturing short-range intent, and a graph-based (*global*) representation capturing dependencies induced by the interaction graph. Since both feed a shared scoring function, they should encode a consistent user state.

We enforce consistency with a redundancy-reduction objective that promotes (i) invariance between the two views of the same user and (ii) low feature redundancy in the aligned space. The latter is important in our setting: without decorrelation, the encoders can agree through a few correlated coordinates, reducing effective capacity and destabilizing optimization. We therefore use the Barlow Twins objective [44], which drives the cross-correlation between paired views toward the identity matrix.

Let h_u denote the local representation from the sequential encoder and e^u the global representation from the graph encoder. For a batch $\mathcal{B}$ of B users, we standardize both embeddings along

the batch dimension (e.g., batch normalization without affine parameters) to obtain $\mathbf{z}_u^{\text{loc}}$ and $\mathbf{z}_u^{\text{glob}}$. We then compute the empirical cross-correlation

$$\mathbf{C}_{ij} = \frac{1}{B} \sum_{u \in \mathcal{B}} z_{u,i}^{\text{loc}} \, z_{u,j}^{\text{glob}}, \qquad i, j \in \{1, \ldots, d\}, \tag{2}$$

where $z_{u,i}^{\text{loc}}$ and $z_{u,j}^{\text{glob}}$ are the i-th and j-th components of the standardized embeddings. The Barlow Twins loss is

$$\mathcal{L}_{\text{BT}} = \sum_{i=1}^{d} \left(1 - \mathbf{C}_{ii}\right)^2 + \lambda \sum_{\substack{i,j=1 \\ i \neq j}}^{d} \mathbf{C}_{ij}^2, \tag{3}$$

where the first term encourages invariance by pushing $\text{diag}(\mathbf{C})$ toward 1, and the second reduces redundancy by pushing off-diagonal correlations toward 0; $\lambda > 0$ controls the trade-off. We add this regularizer to the recommendation objective:

$$\mathcal{L} = \mathcal{L}_{\text{rec}} + w_{BT} \, \mathcal{L}_{\text{BT}}, \tag{4}$$

where $\mathcal{L}_{\text{rec}}$ is the task loss (e.g., next-item prediction / ranking) and w_{BT} controls alignment strength.

3.6 Training process

The essential part in multi-representational setup is to make sure that components do not conflict with each other. Our framework optimizes the weighted loss:

$$L = L_{local} + w_{global} L_{global} + w_{BT} L_{BT}. \tag{5}$$

The first component, L_{local} is the loss function for the sequential model (full cross-entropy in our case) and it serves as the basis. The second term L_{global} is responsible for graph learning and the last one L_{BT} is Barlow Twins for representation alignment.

The last two terms have weights w_{global} and w_{BT} (Eq. 5), respectively, so the influence of the graph model during the training process can be adjusted for each dataset individually.

Our framework relies on the sequential model, enriched by graph representation. To make sure that the graph model provides relevant knowledge, we first let it "warm-up". To do that, during the first N_{warmup} epochs, we train only the graph model. After that, it produces better item embeddings, so we turn on the sequential model and train the whole model. This procedure increases overall performance, which can be seen clearly in the corresponding ablation section. The dynamics of the training process is shown in the Figure 2.

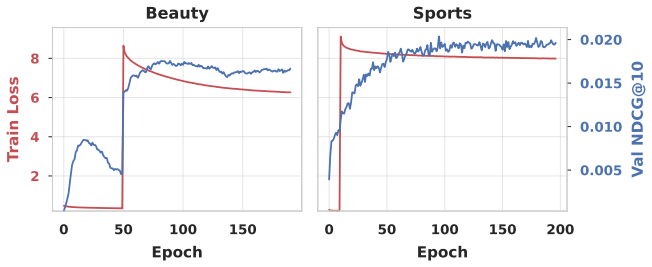

Figure 2: Visualized training process, with training loss and NDCG@10 scales placed on the left and on the right accordingly.

4 Experiments

In this section, we present the main experimental results of our work and describe the evaluation protocol, including the datasets and training setup. We compare our method against standard Transformer-based baselines, recent multi-representational models, and classical non-neural recommenders. The complete results are reported in Table 2.

4.1 Datasets

We benchmark recommendation models on MovieLens-1M[1], three Amazon product categories, and the Yambda music dataset. The statistics of datasets could be seen in the Table 1. For MovieLens and Amazon, we treat an *interaction* as an explicit 1–5 rating associated with a timestamp. For Yambda, we treat an interaction as a timestamped user action on a track (including both implicit and explicit feedback), following the dataset specification.

For e-commerce recommendation, we use the Amazon review datasets [26] (1996–2018) in their 5-core filtered versions, retaining only users and items with at least five interactions; we consider Clothing, Shoes and Jewelry, Sports and Outdoors, and Beauty. Besides ratings and timestamps, Amazon data may include review text and rich item metadata (text, categorical descriptors such as brand and category hierarchy, numerical signals such as price and popularity proxies, and optional related-item links).

Yambda-5B [28] (Yandex Music Billion-interactions DAtaset) is a large-scale open dataset of music consumption collected from Yandex.Music (see Table 1). The log of the dataset includes five event types: *Listen* (implicit), *Like*/*Dislike* (explicit), and *Unlike*/*Undislike* (preference cancellations), with temporal order preserved at up to 5-second granularity. Yambda further provides the `is_organic` flag (organic vs. recommender-driven actions), pretrained audio embeddings for tracks (CNN-based, contrastive training on spectrograms), and track/album/artist metadata. The dataset is released in multiple scale variants (e.g., Yambda-50M, Yambda-500M, Yambda-5B) to support experiments under different computational budgets.

Table 1: Datasets and their statistics used in this work.

Dataset	#Users	#Items	#Interactions
MovieLens-1M	6,040	3,706	1,000,209
Amazon Clothing (5-core)	39,387	23,033	278,677
Amazon Sports (5-core)	35,598	18,357	296,337
Amazon Beauty (5-core)	22,363	12,101	198,502
Yambda-50M (likes)	8283	181,304	881,456

4.2 Evaluation protocol

In this work, we utilize the global time split strategy proposed by [11] to partition each recommendation dataset into training, validation, and test sets. All models, including ours and the baselines, are firstly trained on the training set, and their hyperparameters are tuned using the validation set. Finally, each model is retrained on the combined training and validation datasets using its optimal

[1] https://grouplens.org/datasets/movielens/1m

hyperparameters, and all reported metrics are evaluated on the test dataset.

4.3 Baselines

We compare our CREATE model with other approaches according to *NDCG@K*, *Recall@K* and *Coverage@K* metrics for $K \in \{10, 100\}$ across all datasets. Our model is compared with traditional baselines, such as SVD and Random; two sequential baselines: SASRec [15] and BERT4Rec [32]; three graph models: LightGCN [12], UltraGCN [25] and Self-GNN [23]; the latest multi-representational recommender systems, such as LOOM [21], MRGSRec [2] and GSAU [4]. To ensure a consistent comparison with recent multi-representational methods like LOOM and GSAU, we evaluate SASRec and our CREATE approach in both: our setup and the LOOM setup. In these experiments, SASRec is used as a common baseline to bridge the two setups.

4.4 Performance comparison with state-of-the-art methods

Table 2 demonstrates the comparison between our CREATE approach with diverse set of baselines across five datasets. The gains demonstrate the improvement over strong sequential baseline SASRec [15] with scalable cross-entropy loss [27]. Overall, CREATE consistently improves recommendation quality across most datasets, and the gains remain stable for both cutoffs $K = 10$ and $K = 100$, with significant improvement in **+38%** on NDCG@10 and **+26%** on NDCG@100 on Yambda-50M dataset. We find these results especially encouraging, as they suggest that CREATE can deliver strong improvements on data that better reflects practical deployment scenario.

Across all five datasets, CREATE achieves the strongest ranking quality and recall, indicating consistent advantages over sequential-only (BERT4Rec [32], SASRec [15]), graph-only (LightGCN [12], UltraGCN [25]) and multi-representational (MRGSRec [2]) architectures (see Table 2).

Importantly, CREATE improves upon *each* encoder family represented in the table. Compared to sequential baselines, the gains suggest that augmenting sequence modeling with graph connectivity provides complementary signals beyond next-item dynamics alone. Compared to graph-based baselines, the improvements indicate that incorporating temporal user dynamics strengthens orderless graph representations. As expected, Random and PopRnd obtain very high coverage by design, yet they substantially underperform in ranking quality and recall, emphasizing that high exposure alone does not yield accurate personalization.

Overall, these results support the main premise of CREATE: combining sequential and graph encoders leads to a more expressive representation that can improve the performance of both architectural families, often yielding a favorable accuracy–coverage trade-off.

To compare our approach with the modern multi-representational models in consistent way, we utilized the setup proposed in the work LOOM [21] using the SASRec [27] model as a bridge between two setups. As could be seen in the Table 3, our approach significantly outperforms other multi-representational models, such as LOOM [21], Self-GNN [23] and GSAU [4].

5 Ablation study

In this section, we ablate various components of our model and how they affect the quality of recommendations. The parameters of interest were the number of epochs for warming up the graph encoder, the size of the graph input to our model, various representation alignment methods, and different backbones for the sequential and graph encoders. The results are presented below.

5.1 Graph size

To have a global understanding of users' interactions, we first build user-item graph and then learn it. Evidently, the information contained in the graph influences the resulting performance of the framework. Thus, it is essential to explore the possibilities for further development. In this section, we examine the influence of graph size on overall model performance on different datasets.

The default way to build user-item interactions graph is to take N last user interactions, where N is a hyperparameter for each dataset. In this section, we conduct the experiments by taking different percent $n\%$ of the last interactions per user ranging from 10% to 100%. For each setup, we took the optimal hyperparameter configuration and measured the mean NDCG@10 on four different seeds. Each dataset was split by global temporal split into 80% for the train set and 20% for the test set. In a side-by-side comparison of experiments conducted on MovieLens-1M and Amazon Beauty (Figure 3), it is evident that each dataset has an optimal percent of interactions, leading to graphs with richer structure and better performance. For MovieLens-1M all graphs that cover more than 40% of interactions lead to better performance, while for Amazon Beauty dataset the optimal value is approximately 40%. Lower NDCG@10 scores on graphs with greater amount of interactions may be caused by insufficient number of warm-up epochs, which help to process complex graph structures.

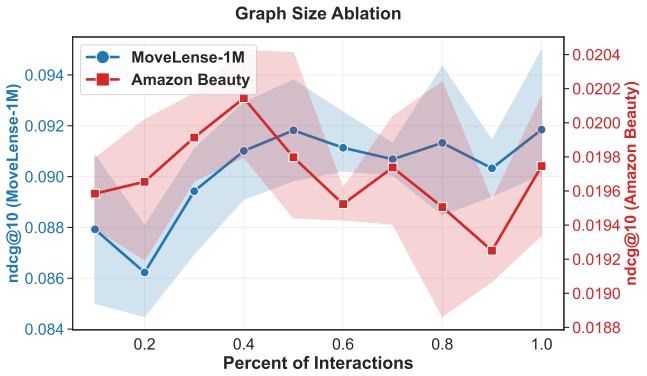

Figure 3: Graph size ablation for MovieLens-1M and Amazon Beauty datasets

5.2 Training with GCN warm-up

The warm-up phase pre-trains the graph encoder so that its representations do not destabilize the sequential encoder during early training. Its required length is data-dependent: it can be skipped on simpler graphs and extended on more structured ones. We start

Table 2: Comparison of models, evaluated using *NDCG@K*, *Recall@K* and *Coverage@K* (in %), with SASRec underlined as main baseline.

Dataset	Metric	SASRec	BERT4Rec	MRGSRec	SVD	Random	PopRnd	LightGCN	UltraGCN	**CREATE**	Gain
Beauty	NDCG@10	1.87 (±0.01)	1.34	0.41	0.52	0.04	0.07	0.93	0.97	2.15 (±0.01)	**+15%**
	Recall@10	3.83 (±0.02)	2.68	0.88	1.04	0.07	0.15	1.88	1.93	4.65 (±0.03)	**+21%**
	Cov@10	70.04 (±1.26)	21.42	3.24	14.02	99.90	91.24	13.27	17.54	75.67 (±1.26)	**+8%**
	NDCG@100	3.73 (±0.02)	3.17	1.27	1.92	0.18	0.26	2.13	2.25	4.25 (±0.03)	**+14%**
	Recall@100	13.67 (±0.08)	12.19	5.38	7.22	0.82	1.21	8.25	8.34	15.64 (±0.13)	**+14%**
	Cov@100	93.46 (±3.29)	54.88	21.80	72.79	100.00	96.85	46.94	47.33	95.37 (±1.04)	**+2%**
Clothing	NDCG@10	0.87 (±0.01)	0.99	0.51	0.57	0.01	0.03	0.78	0.79	1.00 (±0.01)	**+15%**
	Recall@10	1.70 (±0.02)	1.89	1.01	1.43	0.04	0.06	1.62	1.65	2.19 (±0.02)	**+29%**
	Cov@10	52.11 (±3.52)	7.91	5.27	14.09	99.92	94.71	17.50	18.23	57.06 (±3.52)	**+9%**
	NDCG@100	1.86 (±0.02)	2.10	1.36	1.63	0.09	0.13	1.86	1.87	2.19 (±0.01)	**+18%**
	Recall@100	6.88 (±0.03)	8.12	5.50	6.76	0.42	0.65	7.27	7.29	8.37 (±0.04)	**+22%**
	Cov@100	84.55 (±1.75)	31.53	36.17	60.90	100.00	98.15	51.10	52.86	88.67 (±2.05)	**+5%**
Sports	NDCG@10	1.48 (±0.01)	1.35	0.56	0.95	0.04	0.07	1.13	1.19	1.58 (±0.01)	**+7%**
	Recall@10	2.75 (±0.02)	2.55	1.28	1.84	0.07	0.12	2.14	2.21	2.98 (±0.02)	**+8%**
	Cov@10	50.35 (±1.21)	18.36	1.57	18.34	99.96	93.43	17.49	18.23	75.90 (±2.61)	**+50%**
	NDCG@100	3.04 (±0.01)	3.01	1.35	2.32	0.11	0.27	2.55	2.61	3.24 (±0.01)	**+7%**
	Recall@100	10.94 (±0.04)	11.15	5.26	7.53	0.47	1.21	8.43	8.49	11.71 (±0.06)	**+7%**
	Cov@100	85.26 (±0.92)	49.49	11.01	51.32	100.00	98.26	49.60	52.72	96.33 (±0.98)	**+13%**
ML-1M	NDCG@10	7.96 (±0.66)	7.35	1.43	2.78	0.12	0.46	3.21	3.45	8.99 (±0.26)	**+13%**
	Recall@10	13.99 (±1.08)	13.29	3.41	6.21	0.26	0.87	6.57	6.73	15.73 (±0.39)	**+12%**
	Cov@10	45.04 (±1.33)	35.00	6.61	32.10	95.41	60.75	34.13	35.49	40.22 (±0.88)	-10%
	NDCG@100	13.09 (±0.36)	12.35	5.14	5.46	0.70	1.81	5.98	6.17	14.06 (±0.28)	**+7%**
	Recall@100	39.93 (±1.01)	38.67	23.01	18.67	3.50	8.31	20.37	21.19	41.55 (±0.94)	**+4%**
	Cov@100	83.41 (±0.69)	72.31	26.5	73.02	100.00	82.28	57.38	61.40	78.67 (±0.81)	-5%
Yambda-50M	NDCG@10	1.11 (±0.01)	0.39	0.20	0.42	0.0	0.04	1.18	1.19	1.54 (±0.01)	**+38%**
	Recall@10	1.35 (±0.02)	0.87	0.37	0.84	0.0	0.09	1.28	1.30	1.86 (±0.02)	**+37%**
	Cov@10	7.80 (±0.52)	0.006	0.01	5.21	27.61	11.94	10.88	10.99	13.70 (±0.84)	**+75%**
	NDCG@100	1.46 (±0.02)	1.04	0.69	0.71	0.003	0.18	1.21	1.25	1.84 (±0.02)	**+26%**
	Recall@100	3.15 (±0.03)	4.30	2.95	2.19	0.017	0.9	1.43	1.48	3.40 (±0.04)	**+8%**
	Cov@100	19.73 (±0.68)	0.06	0.08	14.58	96.05	16.70	46.11	47.17	60.03 (±1.31)	**+204%**

from the plain sequential setting and sweep the number of warm-up epochs from 0 to 100; we report mean NDCG@10 over 3 seeds using a global temporal split.

We evaluate this procedure on Amazon Beauty and Sports. For each dataset, we measure the best test NDCG@10 over three seeds for warm-up lengths of 0, 10, 20, 50, and 100 epochs. In both datasets, warm-up improves performance over no warm-up, with the best results at the 50 and 10 epochs correspondingly which confirms the choice of the hyperparameter given by Optuna (Fig. 4). Longer warm-up can overfit the graph encoder and reduce end-to-end performance, so the number of warm-up epochs should be tuned.

5.3 Representation alignment

Despite the effectiveness of both sequential and graph recommender systems, they solve different tasks and their representations of data differ from each other. Moreover, while sequential models embed only items [15], the graph model [12] represents both users and items in the vector space. Additionally, as shown in Table 4, the performance of the multi-representational model without graph-sequential alignment is lower than that of the sequential model.

To address these issues, some works—such as [2, 42]—employ contrastive learning, while others propose the Barlow Twins approach [30]. We evaluate our method in three settings: without representation alignment, with the contrastive loss, and with the Barlow Twins loss. Our ablation study shows that the Barlow Twins

Table 3: Model comparison in LOOM [21] setup using *HR*@10 and *NDCG*@10 metrics (in %).

Dataset	Metric	SASRec	GSAU	Self-GNN	LOOM	CREATE
Office	HR@10	1.81	1.95	1.42	2.11	**2.16**
	NDCG@10	0.72	0.78	0.55	0.92	**0.96**
Beauty	HR@10	1.19	1.35	0.98	1.43	**1.49**
	NDCG@10	0.49	0.54	0.40	0.55	**0.57**

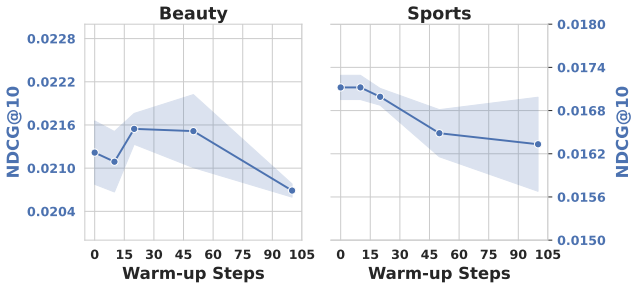

Figure 4: Performance (NDCG@10) vs. the number of GCN warm-up epochs.

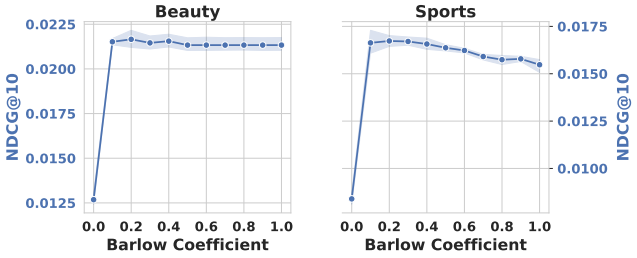

Figure 5: Test ndcg@10 metric by the Barlow coefficient.

approach significantly improves recommendation quality, albeit at the cost of reduced coverage metrics. Across alignment losses, UltraGCN achieves superior performance over LightGCN ones in $K = 10$ quality metrics, whilst the improvements for $K = 100$ are not significant. The benefit of the Barlow Twins approach [30] can be explained by the fact that it does not rely on negative sampling techniques, which often do not fully capture the real data distribution. Moreover, the Barlow Twins approach tends to recommend rarer but relevant items to users, leading to improved recommendation metrics. The correspondence of the test NDCG@10 metric to Barlow Twins coefficient is plotted on Figure 5.

5.4 Using different encoders

Our CREATE approach effectively combines sequential models like SASRec [27] and BERT4Rec [32] with graph-based models like LightGCN [12] and UltraGCN [25]. It outperforms both pure sequential and pure graph baselines (see Tables 2, 4 and 5). This improvement can be explained by the fact that integrating sequential and graph-based patterns provides a richer representation of user behavior than either pattern type alone.

Table 4: Comparison of our model with different alignment losses and the sequential baseline on the Sports and Clothing datasets, using *NDCG@K* and *Coverage@K* metrics (in %).

Sports				
Method	NDCG@10	Cov@10	NDCG@100	Cov@100
Sequential	1.48	50.35	3.04	79.41
CREATE with LightGCN graph encoder				
None	1.37	36.58	2.89	66.95
Contrastive	1.49	**40.16**	3.05	**67.59**
Barlow	**1.56**	25.69	**3.24**	66.02
CREATE with UltraGCN graph encoder				
None	1.38	44.32	2.86	89.76
Contrastive	1.48	60.39	3.07	87.19
Barlow	**1.58**	**75.90**	**3.24**	**96.33**
Clothing				
Method	N@10	C@10	N@100	C@100
Sequential	0.87	52.11	1.86	84.55
CREATE with LightGCN graph encoder				
None	0.90	**77.71**	1.87	**94.37**
Contrastive	0.92	69.53	1.89	89.17
Barlow	**0.95**	72.48	**1.92**	93.02
CREATE with UltraGCN graph encoder				
None	0.92	53.79	2.05	85.34
Contrastive	0.95	48.84	2.10	83.54
Barlow	**1.00**	**57.06**	**2.19**	**87.67**

Moreover, the performance of CREATE improves with the improvement of the underlying sequential and graph encoder quality: the stronger the graph component, the better the overall results. Since our approach works with various sequential and graph encoders, it is highly flexible and can be improved by plugging in enhanced sequential or graph components.

An additional advantage of CREATE is that it does not require user embeddings during inference. Consequently, there is no need to develop folding-in methods for its graph components [41, 42], which simplifies the model pipeline.

6 Conclusion

This paper presents a framework for training recommender models that combines sequential and graph representations of user interactions. This enables the enrichment of vector representations of objects with both local and global information for next-item prediction. A method for training a model within this framework was developed, along with a method for representation alignment between encoders, which, improves the quality of recommendations.

Table 5: Comparison of sequential backbones (SASRec, BERT4Rec) combined with graph encoders (LightGCN, UltraGCN) evaluated by $NDCG@10$ **and** $Recall@10$**.**

Seq. Backbone	Metric	LightGCN	UltraGCN
SASRec	NDCG@10	1.67	1.69
	Recall@10	3.02	3.13
BERT4Rec	NDCG@10	1.39	1.43
	Recall@10	2.65	2.73

Experiments demonstrate improved performance of the proposed method compared to both classical baseline approaches and modern sequential models, as well as recent work in the same field under realistic conditions. An ablation study confirms the influence of various components and parameters provided by the framework on the final quality of recommendations.

References

[1] Vineeta Anand and Ashish Kumar Maurya. 2025. A survey on recommender systems using graph neural network. *ACM Transactions on Information Systems* 43, 1 (2025), 1–49.

[2] Vladimir Baikalov and Evgeny Frolov. 2024. End-to-end graph-sequential representation learning for accurate recommendations. In *Companion Proceedings of the ACM Web Conference 2024*. 501–504.

[3] Tesfaye Fenta Boka, Zhendong Niu, and Rama Bastola Neupane. 2024. A survey of sequential recommendation systems: Techniques, evaluation, and future directions. *Information Systems* 125 (2024), 102427.

[4] Yuwei Cao, Liangwei Yang, Zhiwei Liu, Yuqing Liu, Chen Wang, Yueqing Liang, Hao Peng, and Philip S Yu. 2025. Graph-Sequential Alignment and Uniformity: Toward Enhanced Recommendation Systems. In *Companion Proceedings of the ACM on Web Conference 2025*. 888–892.

[5] Benjamin Paul Chamberlain, Stephen R. Hardwick, David R. Wardrope, Fabon Dzogang, Fabio Daolio, and Saúl Vargas. 2019. Scalable Hyperbolic Recommender Systems. arXiv:1902.08648 [cs.IR] https://arxiv.org/abs/1902.08648

[6] Guoxuan Chen, Lianghao Xia, and Chao Huang. 2025. Lightgnn: Simple graph neural network for recommendation. In *Proceedings of the Eighteenth ACM International Conference on Web Search and Data Mining*. 549–558.

[7] Yingpeng Du, Ziyan Wang, Zhu Sun, Yining Ma, Hongzhi Liu, and Jie Zhang. 2024. Disentangled Multi-interest Representation Learning for Sequential Recommendation. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 677–688.

[8] Chen Gao, Yu Zheng, Nian Li, Yinfeng Li, Yingrong Qin, Jinghua Piao, Yuhan Quan, Jianxin Chang, Depeng Jin, Xiangnan He, et al. 2023. A survey of graph neural networks for recommender systems: Challenges, methods, and directions. *ACM Transactions on Recommender Systems* 1, 1 (2023), 1–51.

[9] Shivangi Gheewala, Shuxiang Xu, and Soonja Yeom. 2025. In-depth survey: deep learning in recommender systems—exploring prediction and ranking models, datasets, feature analysis, and emerging trends. *Neural Computing and Applications* (2025), 1–73.

[10] Danil Gusak, Gleb Mezentsev, Ivan Oseledets, and Evgeny Frolov. 2024. RECE: Reduced Cross-Entropy Loss for Large-Catalogue Sequential Recommenders. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*. 3772–3776.

[11] Danil Gusak, Anna Volodkevich, Anton Klenitskiy, Alexey Vasilev, and Evgeny Frolov. 2025. Time to split: Exploring data splitting strategies for offline evaluation of sequential recommenders. In *Proceedings of the Nineteenth ACM Conference on Recommender Systems*. 874–883.

[12] Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, Yongdong Zhang, and Meng Wang. 2020. Lightgcn: Simplifying and powering graph convolution network for recommendation. In *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval*. 639–648.

[13] Trong Dang Huu Ho and Sang Thi Thanh Nguyen. 2024. Self-Attentive Sequential Recommendation Models Enriched with More Features. In *Proceedings of the 2024 8th International Conference on Deep Learning Technologies (ICDLT)*. 49–55.

[14] Mengyuan Jing, Yanmin Zhu, Tianzi Zang, and Ke Wang. 2023. Contrastive self-supervised learning in recommender systems: A survey. *ACM Transactions on Information Systems* 42, 2 (2023), 1–39.

[15] Wang-Cheng Kang and Julian McAuley. 2018. Self-Attentive Sequential Recommendation. arXiv:1808.09781 [cs.IR] https://arxiv.org/abs/1808.09781

[16] Min-Seung Kim, Yong-Ju Jang, and Tae-Eung Sung. 2025. Graph-based technology recommendation system using GAT-NGCF. *Expert Systems with Applications* (2025), 128240.

[17] TN Kipf. 2016. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907* (2016).

[18] Anton Klenitskiy and Alexey Vasilev. 2023. Turning dross into gold loss: is bert4rec really better than sasrec?. In *Proceedings of the 17th ACM Conference on Recommender Systems*. 1120–1125.

[19] Anton Klenitskiy, Anna Volodkevich, Anton Pembek, and Alexey Vasilev. 2024. Does it look sequential? an analysis of datasets for evaluation of sequential recommendations. In *Proceedings of the 18th ACM Conference on Recommender Systems*. 1067–1072.

[20] Jiacheng Li, Yujie Wang, and Julian McAuley. 2020. Time interval aware self-attention for sequential recommendation. In *Proceedings of the 13th international conference on web search and data mining*. 322–330.

[21] Sijia Li, Min Gao, Zongwei Wang, Yibing Bai, and Wuhan Chen. 2025. Linking Ordered and Orderless Modeling for Sequential Recommendation. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*. 1654–1664.

[22] Jan Malte Lichtenberg, Antonio De Candia, and Matteo Ruffini. 2025. DenseRec: Revisiting Dense Content Embeddings for Sequential Transformer-based Recommendation. *arXiv preprint arXiv:2508.18442* (2025).

[23] Yuxi Liu, Lianghao Xia, and Chao Huang. 2024. Selfgnn: Self-supervised graph neural networks for sequential recommendation. In *Proceedings of the 47th international ACM SIGIR conference on research and development in information retrieval*. 1609–1618.

[24] Tianze Luo, Yong Liu, and Sinno Jialin Pan. 2024. Collaborative sequential recommendations via multi-view gnn-transformers. *ACM Transactions on Information Systems* 42, 6 (2024), 1–27.

[25] Kelong Mao, Jieming Zhu, Xi Xiao, Biao Lu, Zhaowei Wang, and Xiuqiang He. 2021. UltraGCN: ultra simplification of graph convolutional networks for recommendation. In *Proceedings of the 30th ACM international conference on information & knowledge management*. 1253–1262.

[26] J. McAuley. 2016. Amazon Review Data. https://jmcauley.ucsd.edu/data/amazon/

[27] Gleb Mezentsev, Danil Gusak, Ivan Oseledets, and Evgeny Frolov. 2024. Scalable Cross-Entropy Loss for Sequential Recommendations with Large Item Catalogs. In *18th ACM Conference on Recommender Systems (RecSys '24)*. ACM, 475–485. doi:10.1145/3640457.3688140

[28] Alexander Ploshkin, Vladislav Tytskiy, Alexey Pismenny, Vladimir Baikalov, Evgeny Taychinov, Artem Permiakov, Daniil Burlakov, and Eugene Krofto. 2025. Yambda-5B — A Large-Scale Multi-Modal Dataset for Ranking and Retrieval. In *Proceedings of the Nineteenth ACM Conference on Recommender Systems (RecSys '25)*. Association for Computing Machinery, New York, NY, USA, 894–901. doi:10.1145/3705328.3748163

[29] Ahmed Rashed, Shereen Elsayed, and Lars Schmidt-Thieme. 2022. Context and attribute-aware sequential recommendation via cross-attention. In *Proceedings of the 16th ACM conference on recommender systems*. 71–80.

[30] Ivan Razvorotnev, Marina Munkhoeva, and Evgeny Frolov. 2025. Barlow Twins for Sequential Recommendation. *arXiv preprint arXiv:2510.26407* (2025).

[31] Steffen Rendle, Christoph Freudenthaler, Zeno Gantner, and Lars Schmidt-Thieme. 2009. BPR: Bayesian Personalized Ranking from Implicit Feedback. In *UAI*. 452–461.

[32] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer. arXiv:1904.06690 [cs.IR] https://arxiv.org/abs/1904.06690

[33] Xiang Wang, Xiangnan He, Meng Wang, Fuli Feng, and Tat-Seng Chua. 2019. Neural graph collaborative filtering. In *Proceedings of the 42nd international ACM SIGIR conference on Research and development in Information Retrieval*. 165–174.

[34] Ziyang Wang, Huoyu Liu, Wei Wei, Yue Hu, Xian-Ling Mao, Shaojian He, Rui Fang, and Dangyang Chen. 2022. Multi-level contrastive learning framework for sequential recommendation. In *Proceedings of the 31st ACM international conference on information & knowledge management*. 2098–2107.

[35] Shiwen Wu, Fei Sun, Wentao Zhang, Xu Xie, and Bin Cui. 2022. Graph neural networks in recommender systems: a survey. *Comput. Surveys* 55, 5 (2022), 1–37.

[36] Xu Xie, Fei Sun, Zhaoyang Liu, Shiwen Wu, Jinyang Gao, Jiandong Zhang, Bolin Ding, and Bin Cui. 2022. Contrastive learning for sequential recommendation. In *2022 IEEE 38th international conference on data engineering (ICDE)*. IEEE, 1259–1273.

[37] Liangwei Yang, Shengjie Wang, Yunzhe Tao, Jiankai Sun, Xiaolong Liu, Philip S Yu, and Taiqing Wang. 2023. Dgrec: Graph neural network for recommendation with diversified embedding generation. In *Proceedings of the sixteenth ACM*

international conference on web search and data mining. 661–669.

[38] Ji Hyung Yoon and Beakcheol Jang. 2023. Evolution of deep learning-based sequential recommender systems: From current trends to new perspectives. *IEEE Access* 11 (2023), 54265–54279.

[39] Junliang Yu, Xin Xia, Tong Chen, Lizhen Cui, Nguyen Quoc Viet Hung, and Hongzhi Yin. 2023. XSimGCL: Towards extremely simple graph contrastive learning for recommendation. *IEEE Transactions on Knowledge and Data Engineering* 36, 2 (2023), 913–926.

[40] Jun Yuan, Guohao Cai, and Zhenhua Dong. 2025. A Contextual-Aware Position Encoding for Sequential Recommendation. In *Companion Proceedings of the ACM on Web Conference 2025*. 577–585.

[41] Viacheslav Yusupov, Maxim Rakhuba, and Evgeny Frolov. 2025. Ultra Fast Warm Start Solution for Graph Recommendations. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*. 5469–5473.

[42] Viacheslav Yusupov, Nikita Sukhorukov, and Evgeny Frolov. 2026. Efficient incorporation of new interactions in graph recommenders via folding-in: V. Yusupov et al. *User Modeling and User-Adapted Interaction* 36, 1 (2026), 2.

[43] Gholamreza Zare, Nima Jafari, Mehdi Hosseinzadeh, and Amir Sahafi. 2025. DAC-GCN: A Dual Actor-Critic Graph Convolutional Network with Multi-Hop Aggregation for Enhanced Recommender Systems. *Acta Informatica Pragensia* 2025, 3 (2025), 340–364.

[44] Jure Zbontar, Li Jing, Ishan Misra, Yann LeCun, and Stéphane Deny. 2021. Barlow Twins: Self-Supervised Learning via Redundancy Reduction. In *Proceedings of the 38th International Conference on Machine Learning (ICML) (Proceedings of Machine Learning Research, Vol. 139)*. PMLR, 12310–12320. https://proceedings.mlr.press/v139/zbontar21a.html

[45] Peiyan Zhang, Yuchen Yan, Xi Zhang, Chaozhuo Li, Senzhang Wang, Feiran Huang, and Sunghun Kim. 2024. Transgnn: Harnessing the collaborative power of transformers and graph neural networks for recommender systems. In *Proceedings of the 47th International ACM SIGIR conference on research and development in information retrieval*. 1285–1295.

[46] Shengzhe Zhang, Liyi Chen, Chao Wang, Shuangli Li, and Hui Xiong. 2024. Temporal graph contrastive learning for sequential recommendation. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 38. 9359–9367.

[47] Shuai Zhang, Lina Yao, Aixin Sun, and Yi Tay. 2019. Deep learning based recommender system: A survey and new perspectives. *ACM computing surveys (CSUR)* 52, 1 (2019), 1–38.